```
clc;
clear;
close all;

%% ================================================
%% PART 1: VEHICLE PARAMETERS
%% ================================================
m = 1800;               % Vehicle mass (kg)
g = 9.81;               % Gravity (m/s^2)
Crr = 0.01;             % Rolling resistance coefficient
rho = 1.225;            % Air density (kg/m^3)
Cd = 0.29;              % Drag coefficient
A = 2.2;                % Frontal area (m^2)

delta = 0.08;           % Rotational Mass factor
m_eq = m*(1 + delta);

r_wheel = 0.3;          % Wheel radius (m)

%% ================================================
%% PART 2: IMPORT UDDS DRIVE CYCLE
%% ================================================
udds_data = readtable("uddsdc.csv");

t = udds_data.UDDS;       % Time (s)
v_mph = udds_data.Var2;   % Speed (mph)

v = v_mph * 0.44704;      % Convert mph → m/s

dt = mean(diff(t));

a = [diff(v)/dt; 0];      % Acceleration

% Acceleration limits
a_max = 3;
a_min = -4;

a(a>a_max) = a_max;
a(a<a_min) = a_min;

%% ================================================
%% PART 3: LONGITUDINAL FORCES
%% ================================================

gear_ratio = 9;

omega_wheel = v / r_wheel;
omega = omega_wheel * gear_ratio;

F_rr   = m * g * Crr;
F_aero = 0.5 * rho * Cd * A .* v.^2;
F_acc  = m_eq .* a;

theta = deg2rad(2);
```

```
55 F_rg = m*g*sin(theta);
56
57 F_total = F_rr + F_aero + F_acc + F_rg;
58
59 %% ================================================
60 %% PART 4: WHEEL MECHANICAL POWER
61 %% ================================================
62
63 P_wheel = F_total .* v;
64
65 eta_tire = 0.95;
66 P_wheel = P_wheel / eta_tire;
67
68 %% ================================================
69 %% PART 5: MOTOR + REGEN LIMITS
70 %% ================================================
71
72 T_motor_max = 250;
73 T_regen_max = 120;
74 P_motor_max = 80e3;
75
76 v_min_regen = 1.5;
77
78 eta_drivetrain = 0.95;
79
80 P_mech = zeros(size(P_wheel));
81
82 omega_base = P_motor_max / T_motor_max;
83
84 for i = 1:length(P_wheel)
85
86     if P_wheel(i) > 0
87
88         if omega(i) <= omega_base
89             T_avail = T_motor_max;
90         else
91             T_avail = P_motor_max / omega(i);
92         end
93
94         P_avail = T_avail * omega(i);
95
96         P_mech(i) = min(P_wheel(i), P_avail);
97
98     elseif P_wheel(i) < 0 && v(i) > v_min_regen
99
100        P_regen_limit = T_regen_max * omega(i);
101
102        P_mech(i) = -min(abs(P_wheel(i)), P_regen_limit);
103
104    else
105
106        P_mech(i) = 0;
107
108    end
```

```
109
110 end
111
112 % drivetrain efficiency
113 P_mech = P_mech / eta_drivetrain;
114
115 %% ================================================
116 %% PART 6: DISTANCE & IDEAL ENERGY
117 %% ================================================
118
119 E_Wh = trapz(t, P_mech) / 3600;
120
121 dist_km = trapz(t, v) / 1000;
122
123 E_Wh_per_km = E_Wh / dist_km;
124
125 %% ================================================
126 %% PART 7: MOTOR + INVERTER EFFICIENCY
127 %% ================================================
128
129 omega_max = max(omega) + eps;
130
131 drive_eff = ones(size(P_mech));
132 regen_eff = ones(size(P_mech));
133
134 for i = 1:length(P_mech)
135
136     if P_mech(i) > 0
137
138         T_req = P_mech(i) / max(omega(i),eps);
139
140         Tn = abs(T_req) / T_motor_max;
141
142         on = omega(i) / omega_max;
143
144         drive_eff(i) = 0.90 ...
145             - 0.15*(1-on)^2 ...
146             - 0.20*(Tn-0.6)^2;
147
148         drive_eff(i) = min(max(drive_eff(i),0.75),0.92);
149
150     elseif P_mech(i) < 0
151
152         T_req = abs(P_mech(i)) / max(omega(i),eps);
153
154         Tn = T_req / T_regen_max;
155
156         on = omega(i) / omega_max;
157
158         regen_eff(i) = 0.75 ...
159             - 0.20*(1-on)^2 ...
160             - 0.30*(Tn-0.5)^2;
161
162         regen_eff(i) = min(max(regen_eff(i),0.55),0.80);
```

```
163
164     end
165
166 end
167
168 %% ================================================
169 %% PART 8: BATTERY POWER FLOW
170 %% ================================================
171
172 P_batt_disch_max = 90e3;
173 P_batt_char_max  = 40e3;
174
175 P_accessory = 900;
176
177 Batt_capacity = 50; % kWh
178
179 P_drive = zeros(size(P_mech));
180 P_regen = zeros(size(P_mech));
181
182 for i = 1:length(P_mech)
183
184     if P_mech(i) > 0
185
186         P_drive(i) = P_mech(i) / drive_eff(i);
187
188         P_drive(i) = min(P_drive(i), P_batt_disch_max);
189
190     elseif P_mech(i) < 0
191
192         P_regen(i) = abs(P_mech(i)) * regen_eff(i);
193
194         P_regen(i) = min(P_regen(i), P_batt_char_max);
195
196     end
197
198 end
199
200 P_batt = P_drive - P_regen + P_accessory;
201
202 %% ================================================
203 %% PART 9: SOC MODEL
204 %% ================================================
205
206 SOC = zeros(size(t));
207
208 SOC(1) = 0.80;
209
210 for i = 2:length(t)
211
212     E_step_Wh = P_batt(i) * dt / 3600;
213
214     SOC(i) = SOC(i-1) - E_step_Wh/(Batt_capacity*1000);
215
216 end
```

```
217
218 %% ================================================
219 %% PART 10: PLOTS
220 %% ================================================
221
222 figure('Name','EV UDDS Simulation');
223
224 subplot(3,1,1)
225 plot(t,v*3.6,'r','LineWidth',1.5)
226 xlabel('Time (s)')
227 ylabel('Speed (km/h)')
228 title('UDDS Drive Cycle')
229 grid on
230
231 subplot(3,1,2)
232 plot(t,P_drive/1000,'b'); hold on
233 plot(t,-P_regen/1000,'g')
234 xlabel('Time (s)')
235 ylabel('Power (kW)')
236 legend('Drive','Regen')
237 title('Battery Power Flow')
238 grid on
239
240 subplot(3,1,3)
241 plot(t,SOC*100,'k','LineWidth',1.5)
242 xlabel('Time (s)')
243 ylabel('SOC (%)')
244 title('Battery State of Charge')
245 grid on
246
247 %% ================================================
248 %% PART 11: FINAL RESULTS
249 %% ================================================
250
251 E_drive_Wh = trapz(t,P_drive)/3600;
252
253 E_regen_Wh = trapz(t,P_regen)/3600;
254
255 E_batt_net = E_drive_Wh - E_regen_Wh;
256
257 E_brake_mech = trapz(t,abs(min(P_mech,0)))/3600;
258
259 regen_percentage = (E_regen_Wh/E_brake_mech)*100;
260
261 P_drive_max = max(P_drive)/1000;
262
263 P_regen_max = max(P_regen)/1000;
264
265 P_wheel_max = max(P_mech)/1000;
266
267 SOC_initial = SOC(1)*100;
268
269 SOC_final = SOC(end)*100;
270
```

```
271 SOC_delta = SOC_initial - SOC_final;
272
273 avg_speed_kmh = mean(v)*3.6;
274
275 %% ================================================
276 %% PRINT RESULTS
277 %% ================================================
278
279 fprintf('\nVEHICLE & DRIVE CYCLE\n\n');
280
281 fprintf('Vehicle Mass\n: %.0f kg\n\n',m);
282
283 fprintf('Total Distance Travelled\n: %.3f km\n\n',dist_km);
284
285 fprintf('Average Speed\n: %.2f km/h\n\n',avg_speed_kmh);
286
287 fprintf('Drive Cycle Duration\n: %.0f s\n\n',t(end));
288
289
290 fprintf('ENERGY CONSUMPTION\n\n');
291
292 fprintf('Wheel Mechanical Energy\n: %.2f Wh\n\n',E_Wh);
293
294 fprintf('Battery Energy Drawn\n: %.2f Wh\n\n',E_drive_Wh);
295
296 fprintf('Battery Energy Regenerated\n: %.2f Wh\n\n',E_regen_Wh);
297
298 fprintf('Net Battery Energy Used\n: %.2f Wh\n\n',E_batt_net);
299
300 fprintf('Specific Energy Consumption\n: %.2f Wh/km\n\n',E_batt_net/dist_km);
301
302
303 fprintf('POWER PERFORMANCE\n\n');
304
305 fprintf('Maximum Wheel Power\n: %.2f kW\n\n',P_wheel_max);
306
307 fprintf('Maximum Battery Drive Power\n: %.2f kW\n\n',P_drive_max);
308
309 fprintf('Maximum Regen Power\n: %.2f kW\n\n',P_regen_max);
310
311
312 fprintf('REGENERATIVE BRAKING\n\n');
313
314 fprintf('Mechanical Braking Energy\n: %.2f Wh\n\n',E_brake_mech);
315
316 fprintf('Regen Energy Recovery\n: %.2f %%\n\n',regen_percentage);
317
318
319 fprintf('BATTERY STATE OF CHARGE\n\n');
320
321 fprintf('Initial SOC\n: %.2f %%\n\n',SOC_initial);
322
323 fprintf('Final SOC\n: %.2f %%\n\n',SOC_final);
324
```

```
325 fprintf('Net SOC Change\n: %.3f %%\n\n',SOC_delta);
326
327
328 %% ================================================
329 %% MOTOR TORQUE SPEED CHARACTERISTIC
330 %% ================================================
331
332 omega_range = linspace(0, max(omega)*1.2, 500);
333
334 T_motor = zeros(size(omega_range));
335
336 for i = 1:length(omega_range)
337
338     if omega_range(i) <= omega_base
339         T_motor(i) = T_motor_max;
340     else
341         T_motor(i) = P_motor_max / omega_range(i);
342     end
343
344 end
345
346 figure
347
348 plot(omega_range, T_motor,'LineWidth',2);
349
350 xlabel('Motor Speed (rad/s)');
351 ylabel('Torque (Nm)');
352 title('EV Motor Torque-Speed Curve');
353
354 grid on;
355
356 %% ================================================
357 %% MOTOR EFFICIENCY MAP
358 %% ================================================
359
360 speed_norm = omega/omega_max;
361
362 torque = P_mech ./ max(omega,eps);
363
364 torque_norm = abs(torque)/T_motor_max;
365
366 figure
367
368 scatter(speed_norm, torque_norm, 15, drive_eff,'filled')
369
370 colorbar
371
372 xlabel('Normalized Speed');
373 ylabel('Normalized Torque');
374
375 title('Motor Efficiency Map');
376
377 grid on;
378
```

```
379 %% ================================================
380 %% ENERGY FLOW ANALYSIS
381 %% ================================================
382
383 E_accessory = trapz(t, ones(size(t))*P_accessory)/3600;
384
385 fprintf('\nENERGY FLOW ANALYSIS\n\n' );
386
387 fprintf('Drive Energy From Battery : %.2f Wh\n',E_drive_Wh);
388
389 fprintf('Recovered Regen Energy    : %.2f Wh\n',E_regen_Wh);
390
391 fprintf('Accessory Consumption     : %.2f Wh\n',E_accessory);
392
393 fprintf('Net Battery Consumption   : %.2f Wh\n',E_batt_net);
394
395
396
397 %% ================================================
398 %% RANGE ESTIMATION
399 %% ================================================
400
401 usable_SOC = 0.8 - 0.2;   % 80% to 20%
402
403 usable_energy = Batt_capacity * usable_SOC * 1000;% Wh
404
405 range_est = usable_energy / (E_batt_net/dist_km);
406
407 fprintf('\nESTIMATED EV RANGE\n\n');
408
409 fprintf('Estimated Range (UDDS Based) : %.1f km\n',range_est);
410
411
412
```